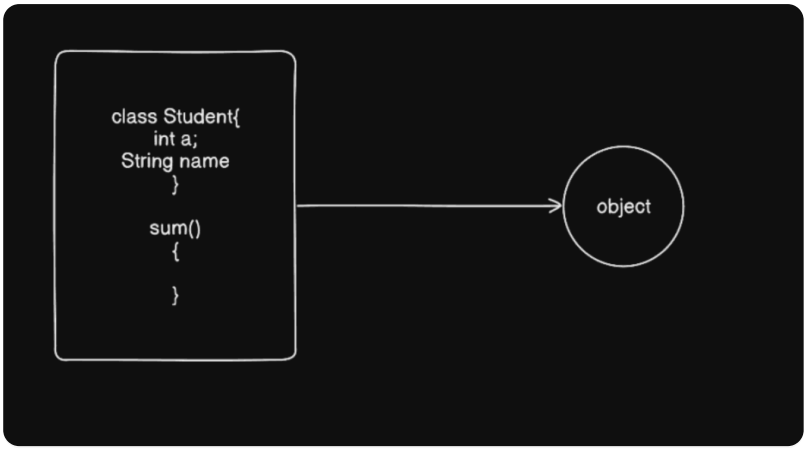
# OOPs

27 June 2026 11:05

## What is OOPs ?

> **O**bject **o**riented **p**rogramming

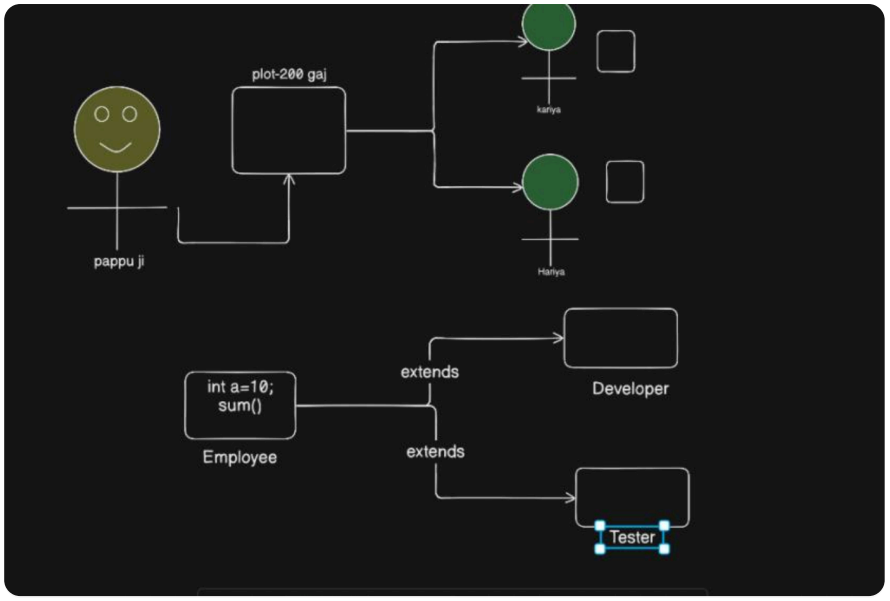



`What is class ?` `What is object ?`

**Q If I say Employee, Manager, Tester, Developer are all of them completely different ?**

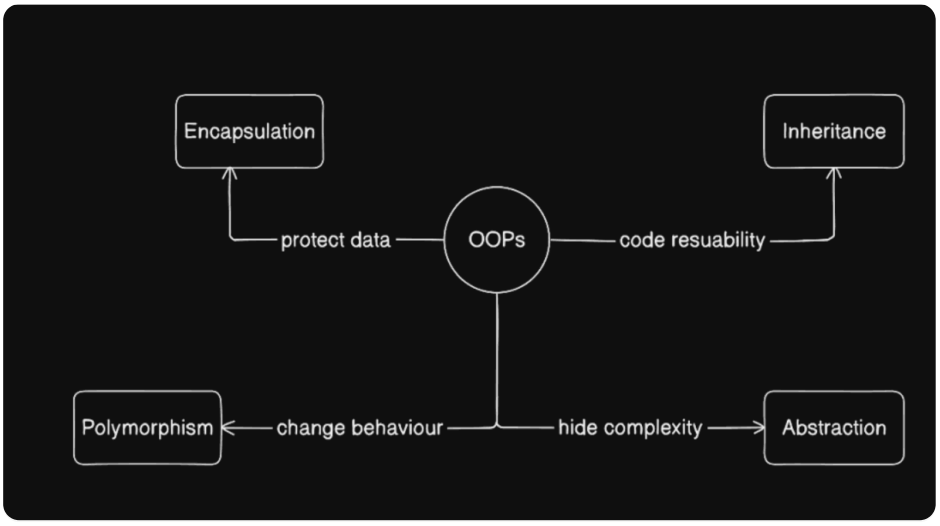
**A No. All of these are employees**

> **Note:** In java common things can be placed in **parent class**. Specific things can be placed in **child class**




## There are 4 principles in OOPs

| | |
|---|---|
| 1 | Inheritance |
| 2 | Encapsulation |
| 3 | Polymorphism |
| 4 | Abstraction |



# 02 Encapsulation

> **Encapsulation** means binding data and methods together inside a class

**Encapsulation = data hiding + controlled access**

EG:

ATM machine does not give direct access to bank database.
We use buttons and screen options; that is controlled access.




- **Balance is private**
- Outside class cannot directly change balance, money can be added only through `deposit()`.

# 01 Inheritance:

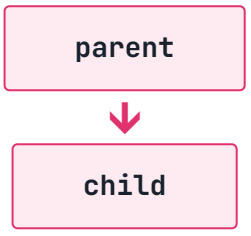
> Child class gets properties and behaviour from parent class

- We use `extends` keyword to make child-parent relationship

### Types of inheritance

#### 1. Single inheritance

one parent , one child

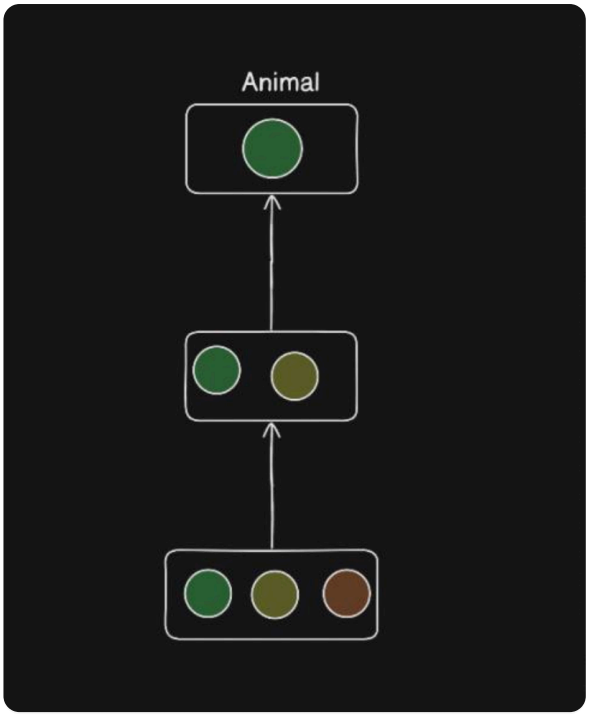


## 2. Multilevel inheritance

Parent to child to grandchild

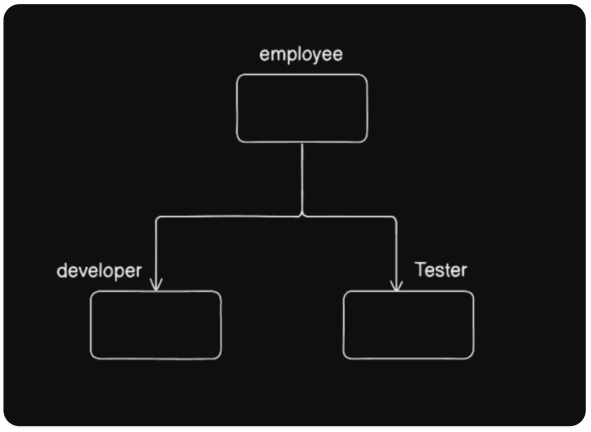
`Animal` -----> `Dog`

`Dog` -----> `Puppy`




## 3. Hierarchical inheritance

One parent, multiple children

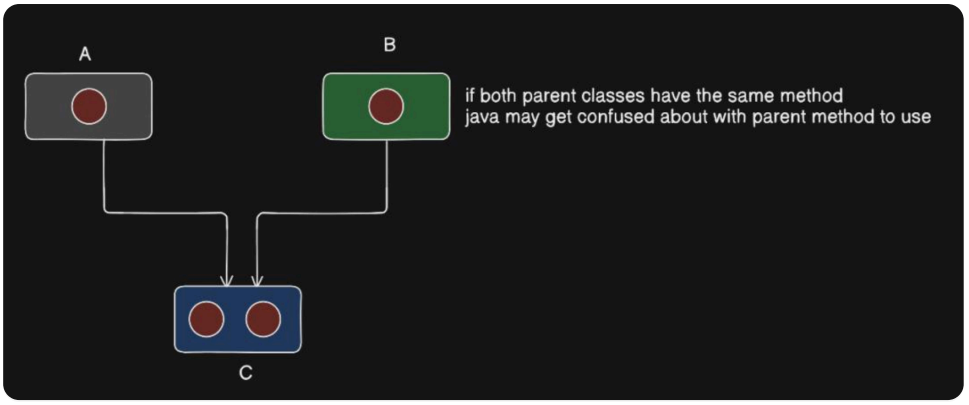



## 4. Multiple inheritance with classes is not allowed

Java does not allow

```
class C extends A, B
{


}
```




## What is `this` keyword ?

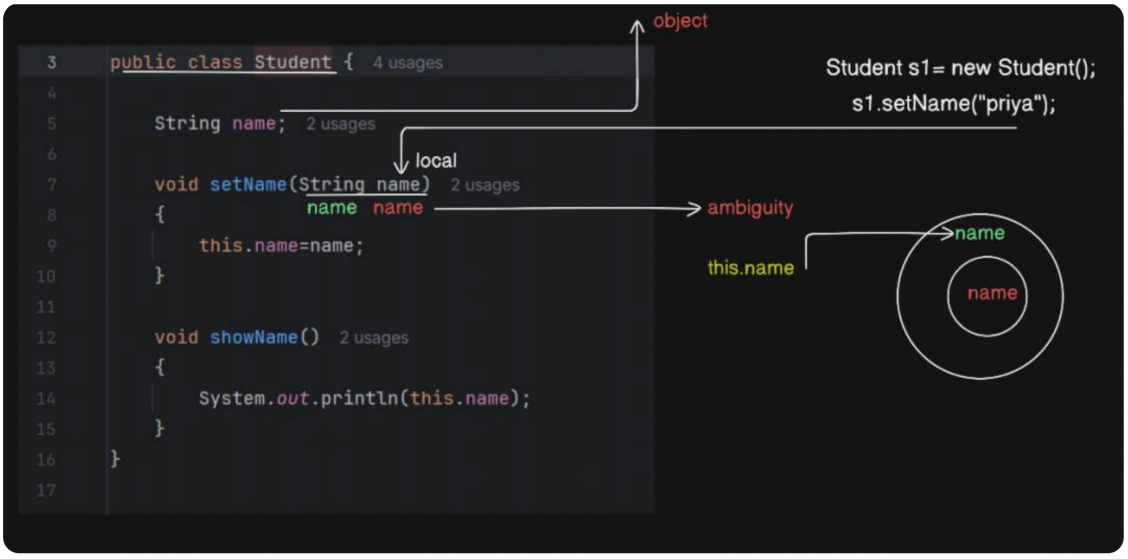
> `this` keyword is a reference variable that refers to the current object

SIMPLE :

This means the current object on which the code is running.

SIMPLE:

When one object calls a method, inside that method `this` points to that same object.



### Why we need this ?

**When local variable name and instance variable name are same, `this` is used to refer to the instance variable**

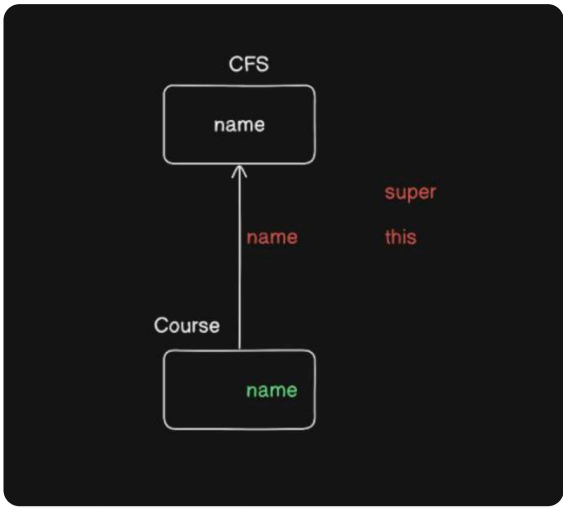
### Uses of this:

| | |
|---|---|
| 1 To access current class variable | 2 To call current class method |
| 3 To call current class constructor | 4 To pass current object as an argument |
| 5 To return current object | |

## What is super keyword ?

`super` is a reference variable used inside a child class.

`super` refers to the immediate parent class part of the current object

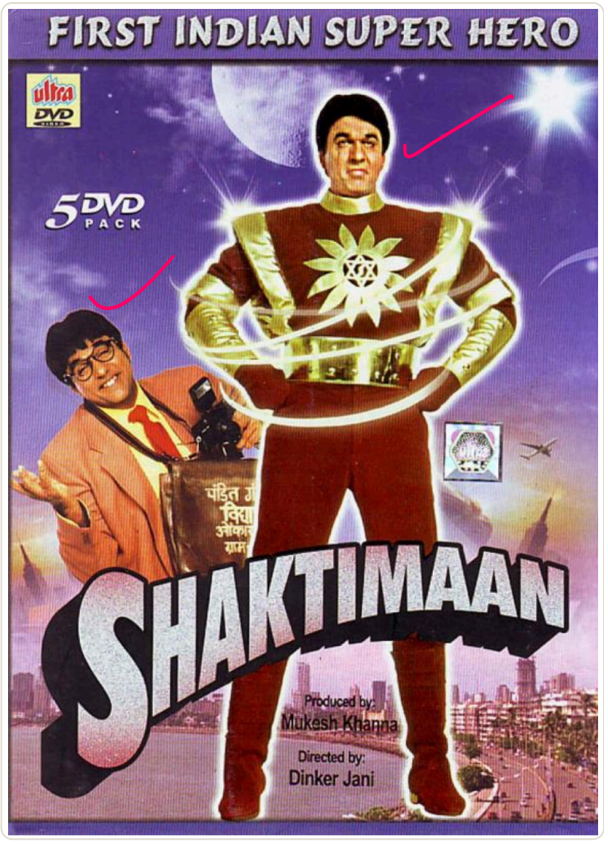



## 03 Polymorphism

**Polymorphism** means one thing can behave in multiple forms

**Polymorphism = same name, different behaviours**






### Real world examples

The word "pay" can mean

1. **Pay by cash**
2. **Pay by UPI**
3. **Pay by card**

### In java polymorphism mainly has two types

1. **Method Overloading**
2. **Method Overriding**

### Method Overloading

> Same method name, different parameters, **same class**.



```java
class Calculator  no usages
{
    int add(int a, int b)  no usages
    {
        return a+b;
    }

    int add(int a, int b,int c)  no usages
    {
        return a+b+c;
    }

    double add(double a, double b)  no usages
    {
        return a+b;
    }
}
```

### Method Overriding

> Same method name, same parameters, **parent-child relationship**.

```java
class Payment  1 usage  1 inheritor
{
    void pay()  1 usage
    {
        System.out.println("Generic Payment");
    }
}

class UpiPayment extends Payment  2 usages
{
    @Override  2 usages
    void pay() {
        System.out.println("Payment done using UPI");
    }


public class PaymentTest {
    public static void main(String[] args) {
        UpiPayment upiPayment = new UpiPayment();
        upiPayment.pay();
    }
}
```

### Dynamic Method Dispatch:

| Runtime polymorphism : | Compile time polymorphism |
| --- | --- |

## 04 Abstraction

> **Abstraction** means showing necessary details and hiding internal complexity